

SuretySeven — Engineering Take-Home Assignment

Background

SuretySeven is building technology that helps insurers and their distribution partners process surety bond applications.

For this assignment, assume that a broker submits a bond application containing:

- Applicant/company information
- Bond amount
- Bond type
- Effective date
- Obligee information

The application needs to be evaluated before an underwriting decision can be made.

As part of the evaluation, the system may need to retrieve additional information from external systems.

Your task is to build a small but production-minded version of this workflow.

Objective

Build a service that can:

1. Accept a new bond application.
2. Validate the application.
3. Retrieve additional applicant information from an external API.
4. Calculate a simple underwriting score.
5. Produce an underwriting decision.
6. Expose the application and its current status through an API.
7. Handle failures from external dependencies gracefully.
8. Notify a downstream system when the decision is completed.

You may choose any programming language, framework, database, cloud platform, or architecture.

You are also free to use AI coding/design tools.

The goal is not to build a production-ready insurance platform. We want to understand how you approach a real-world engineering problem.

Functional Requirements

1. Create Application

Provide an API that accepts a request similar to:

```
{
  "applicantId": "COMP-123",
  "bondType": "CONTRACT",
  "bondAmount": 500000,
  "effectiveDate": "2026-10-01",
  "obligee": {
    "name": "ABC Construction LLC"
  }
}
```

The API should:

- Validate the request.
 - Generate an application ID.
 - Persist the application.
 - Return the application ID and initial status.
-

2. Applicant Information

The application requires additional information from an external Applicant API.

For example:

```
GET /external/applicants/{applicantId}
```

The response may contain:

```
{
  "applicantId": "COMP-123",
  "annualRevenue": 12000000,
  "yearsInBusiness": 8,
  "creditScore": 760,
  "existingExposure": 1500000
}
```

You should provide a mock implementation of this external dependency.

The external API should be capable of simulating:

- Successful responses
- Timeout
- 5xx failure
- Invalid/malformed response
- Slow response

3. Underwriting Decision

Implement a simple deterministic scoring model.

You may define your own scoring rules, but document them clearly.

For example:

Condition	Score
Credit score $\geq$ 750	+30
Credit score 700–749	+20
Credit score $<$ 700	+5
Years in business $\geq$ 5	+20
Years in business $<$ 5	+10
Bond amount $\leq$ 10% of annual revenue	+30
Bond amount $>$ 10% of annual revenue	+10
Existing exposure $<$ 20% of annual revenue	+20
Existing exposure $\geq$ 20%	+5

Example decision:

Score $\geq$ 80 → APPROVE
Score 50–79 → REFER
Score $<$ 50 → DECLINE

The scoring rules should be implemented so that they can reasonably be changed later.

4. Application Status

Expose an API such as:

```
GET /applications/{applicationId}
```

The response should contain at least:

```
{
  "applicationId": "APP-123",
  "status": "APPROVED",
  "score": 87,
  "decision": "APPROVE",
  "createdAt": "...",
  "updatedAt": "..."
}
```

Consider what information should be retained to explain how the decision was reached.

5. Downstream Notification

When an application reaches a final decision, send an event/message to a downstream system.

For example:

```
{
  "eventType": "APPLICATION_DECISIONED",
  "applicationId": "APP-123",
  "decision": "APPROVE",
  "score": 87
}
```

You may implement this using:

- A message queue
- Pub/sub
- Kafka
- RabbitMQ
- An HTTP callback
- An in-process mechanism

Choose what you think is appropriate and explain your choice.

The downstream system should also be mocked.

Non-Functional Requirements

The system should demonstrate reasonable consideration for:

Reliability

What happens if:

- The Applicant API times out?
- The Applicant API returns 500?
- The downstream notification fails?
- The same application request is submitted twice?
- The service crashes halfway through processing?

You don't need to solve every possible distributed-systems problem, but your design should explicitly address the important ones.

Idempotency

Assume that clients may retry requests.

Demonstrate how you prevent duplicate applications or duplicate downstream events.

Observability

Provide useful:

- Logs
- Error information
- Application/request correlation IDs

Metrics are optional but encouraged.

Security

Consider:

- Input validation
- Authentication/authorization assumptions
- Sensitive information in logs
- API abuse

A full authentication system is not required.

Testing

Provide automated tests covering important business logic and failure scenarios.

User Interface

A UI is **optional**.

If you build one, it should at minimum allow a user to:

1. Submit an application.
2. View its status.
3. View the resulting decision and score.

Do not spend most of the assignment building a polished UI.

We are primarily evaluating engineering judgment and implementation quality.

Architecture

Include a short architecture document containing:

1. Architecture diagram.
2. Major components.
3. Data flow.
4. API design.
5. Persistence model.
6. Failure-handling strategy.
7. Idempotency strategy.
8. Key architectural trade-offs.
9. What you would change if this system had to handle 100x the current traffic.

The architecture document should be concise. Approximately 3–5 pages is sufficient.

AI Usage

You are explicitly allowed to use AI tools, including:

- ChatGPT
- Claude
- GitHub Copilot
- Cursor
- Kiro
- Other coding agents

We are not evaluating whether you can write every line of code manually.

However, you should be able to explain:

- What you asked AI to generate.
- What you changed.
- What design decisions you made yourself.
- Where AI-generated code was incorrect, incomplete, or inappropriate.

Include a short `AI_USAGE.md` describing how AI was used.

Deliverables

Submit a Git repository containing:

```
/README.md
/AI_USAGE.md
/src
/tests
/docs
  architecture.md
  architecture.png
```

The README should contain:

- Problem interpretation
- How to run the application
- How to run tests
- API documentation
- Example requests
- Architectural decisions
- Known limitations
- What you would do next with another 2 weeks

The application should preferably be runnable with a single command, for example:

```
docker compose up
```

or an equivalent simple setup.

Time Constraint

This assignment is designed to be completed alongside a normal full-time job.

Please spend approximately **15–25 hours total** on it over a maximum period of **two weeks**.

Do not build unnecessary features.

We value a well-designed, well-tested small system more than a large unfinished system.

Evaluation

We will primarily evaluate:

1. Engineering judgment

Can you distinguish what is important from what is unnecessary?

2. Architecture

Does the architecture make sense for the problem?

Are the trade-offs understood?

3. Reliability

How does the system behave when dependencies fail?

4. Code quality

Is the code understandable, maintainable, and appropriately structured?

5. API & data modeling

Are the interfaces and persistence model thoughtfully designed?

6. Testing

Do the tests provide meaningful confidence?

7. Product thinking

Does the implementation consider how an actual user/operator would use and troubleshoot the system?

8. Communication

Can you clearly explain your decisions and trade-offs?

9. AI-assisted development

Can you effectively use AI without blindly trusting generated code?

Important

There is no single correct architecture.

For example, a synchronous REST implementation backed by PostgreSQL can be a perfectly good solution.

A serverless/event-driven implementation using queues can also be a good solution.

We care more about whether the architecture is **appropriate for the problem and clearly justified** than whether it uses a particular technology.